AstroSite Finder (siteplanning)

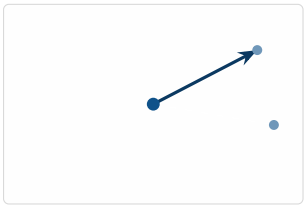
Adelaide Night-Sky Site Selection and Field Planning

Simon Knight

Adelaide, Australia

March 2026 • Version 0.1.0

Last updated: March 11, 2026



Abstract. AstroSite Finder is a client-side, map-centric decision aid for astrophotography site selection around Adelaide. It ranks candidate locations for time-bound night-sky targets (lunar eclipse, Milky Way core, aurora, meteor showers) under practical constraints including light pollution, city-glow directionality, equipment configuration, and local horizon obstruction. The system combines heuristic scoring with interactive visualization in Mapbox GL, event geometry derived from astronomy-engine, and field-execution utilities such as Street View alignment, horizon vs trajectory plots, a live radar overlay, and itinerary export for mobile navigation.

Contents

List of Design Decisions & Rules

Design Decision 1: Report Update Notes

- Convert the report to non-draft style for distribution.
- Add a short section describing the expected schema for `public/discovered.json`.
- Consider adding a score-breakdown table to match the on-screen heuristics.

1 Problem Framing and Goals

AstroSite Finder is designed to reduce the time from *intent* ("I want to shoot an event tonight") to *execution* ("I have a ranked shortlist of locations and a concrete plan"). The key observation is that astrophotography location choice is both geometric and operational: an event has a time-varying azimuth/altitude, but a usable plan must also account for city glow directionality, terrain obstructions, equipment constraints, and logistics.

Design Decision 2: Design Principle: Explainable Heuristics

The system favors transparent, inspectable scoring heuristics over opaque optimization. A score is only useful if the photographer can understand why a site ranks highly and can adjust parameters to match the actual intent. In practice this means: (i) a small number of score terms, (ii) parameters that map to real actions (change azimuth, change target class, change lens), and (iii) deterministic behavior.

Non-goals include server-side persistence, user accounts, and physically rigorous sky modeling. Instead, the system targets a practical decision aid that is robust under incomplete data and works end-to-end as a client-only Single-Page Application (SPA). This constraint materially shapes the architecture: event geometry, site scoring, and derived visualizations must be computable in-browser within interactive latency, and the UI must tolerate missing optional data sources.

2 System Overview

The application is a Vite + React + TypeScript SPA. It runs entirely in the browser and integrates three external capability classes:

1. Map rendering and terrain elevation sampling via Mapbox GL JS.
2. Astronomy event estimation via `astronomy-engine`.
3. Visualization primitives via Recharts (charts) and small UI utilities (QR codes, icons).

The main entry point is `src/main.tsx`, which mounts the React root. The bulk of the system is implemented in `src/App.tsx` with two supporting core modules: `src/core/astronomy.ts` and `src/core/evaluator.ts`.

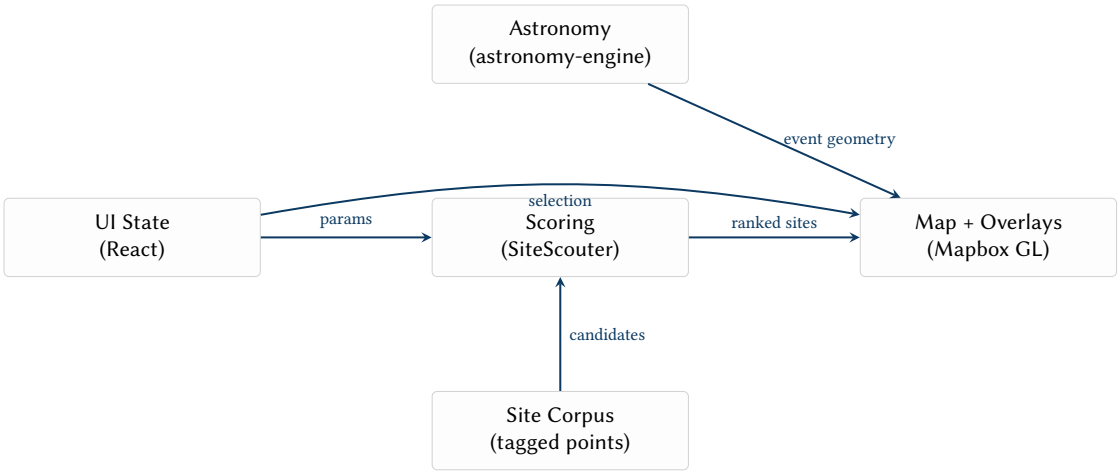


Figure 1. End-to-end information flow in AstroSite Finder.

: Client-Only Execution

All primary workflows (site ranking, event display, map overlay, itinerary export) must complete without a backend. Optional enhancements may load local static data (e.g. `discovered.json`), but must degrade gracefully.

3 Architecture and Data Flow

3.1 Inputs and Outputs

Inputs. User-selected parameters include event type (eclipse / Milky Way / aurora / meteor), target object type, target azimuth, and equipment parameters (lens focal length proxy, aperture, crop factor, megapixels). The system also consumes a local site corpus (tagged locations) and may optionally ingest additional sites from `public/discovered.json`. The parameters are persisted in localStorage to support repeated scouting sessions: users commonly iterate azimuth and lens choice while keeping other preferences stable.

Outputs. The primary output is a ranked shortlist of sites (top N) with visual map markers. Secondary outputs include: (i) event arrows (azimuth bearings and framing guides), (ii) a horizon obstruction profile with an overlaid object trajectory, (iii) a darkness timeline, (iv) external scouting links, and (v) a clipboard-exportable itinerary. These outputs are intentionally heterogeneous: some are "decision" artifacts (rank list), others are "execution" artifacts (QR + itinerary), and others are "sanity checks" (horizon vs altitude).

Figure 2. User workflow pipeline: intent → ranking → inspection → execution.

3.2 Processing Pipeline

The core computational loop is:

1. Construct scouting parameters from UI state.
2. Rank candidate sites using `SiteScouter.rankSites`.
3. When a site is selected, compute event geometry using `getUpcomingEvents`.
4. Render overlays: markers, arrows, and optional radar tiles.
5. Optionally sample terrain elevations along rays to estimate horizon obstruction.

> **Design Decision 3: Directional Scoring**
>
> Unlike static "darkness" rankings, the system explicitly accounts for viewing direction: looking into the city dome is penalized using an angular separation model between target azimuth and bearing to the Adelaide Central Business District (CBD).

4 Core Modules

4.1 Astronomy and Light Heuristics

The module `src/core/astronomy.ts` provides two classes of functions:

1. **Sky quality and city glow:** `estimateBortle` buckets distance-from-CBD into a coarse Bortle estimate; `cityLightImpact` maps directional alignment with the city into a scalar impact value.
2. **Event geometry and timeline:** `getUpcomingEvents` computes approximate azimuth/altitude for an eclipse (when available), a Milky Way core proxy, a simplified aurora direction, and a meteor radiant. It also constructs a darkness timeline using rise/set and altitude searches.

The module is deliberately resilient: when external search functions fail, it falls back to a fixed event instance to preserve UI continuity. This is a pragmatic choice: for a field-planning tool, it is preferable to show a plausible trajectory and continue rendering UI affordances (arrows, charts, timelines) than to present a blank state.

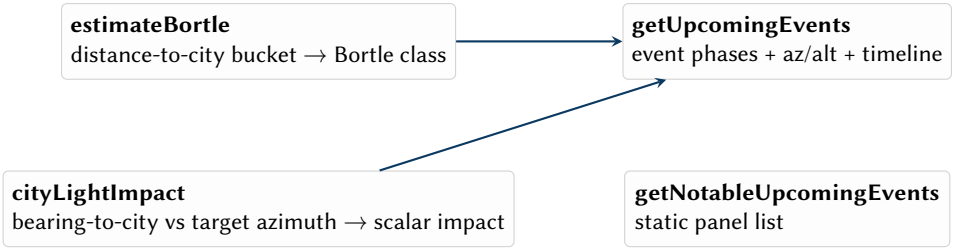


Figure 3. Astronomy module responsibilities and how they feed the UI.

4.2 Scoring and Ranking

The module `src/core/evaluator.ts` defines the site domain model (`Site`) and implements `SiteScouter`. A site score is composed from:

1. Light pollution (Bortle-derived) weighted by target type.

2. City orientation score penalizing looking toward Adelaide.
3. Equipment viability bonuses inferred from tags and focal length.
4. Seasonality heuristics (e.g. Milky Way core season windows).
5. Feature bonuses based on semantic tags.

Axiom 1 (Explainable Score)**.** *All score components must be expressible as a small set of terms with clear interpretation (pollution, directionality, features). This ensures the UI can surface a breakdown without reverse engineering.*

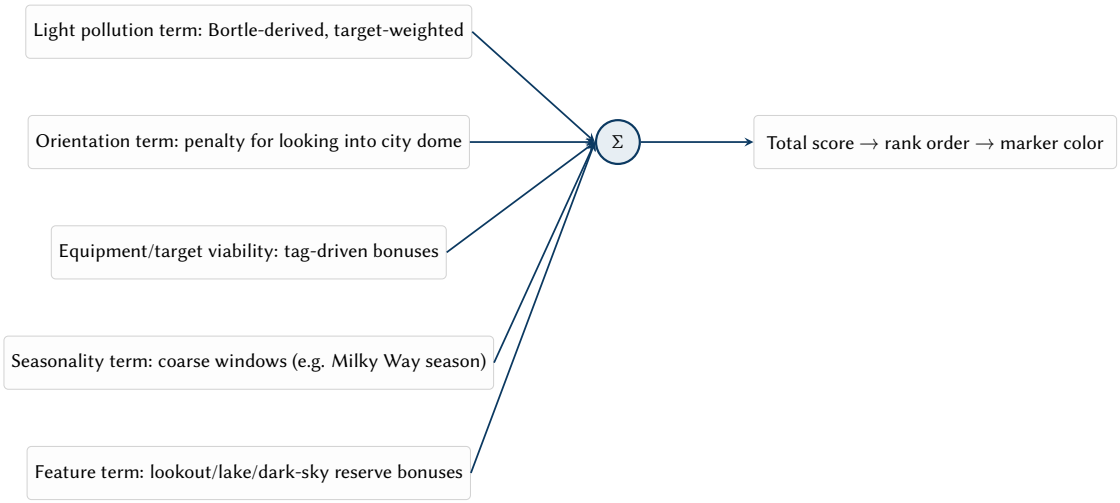


Figure 4. Conceptual structure of the heuristic site score.

4.3 Site Corpus

The baseline candidate set is defined in `src/data/sites.ts`. Sites are tagged (lookout, lake, coast, dark-sky reserve, etc.) and tags act as the primary semantic interface between the corpus and the scorer. Tags are an intentionally lightweight ontology: they do not attempt to encode everything about a location, but they capture compositional affordances that meaningfully affect astrophotography outcomes (foreground opportunities, horizon openness, altitude, and perceived darkness).

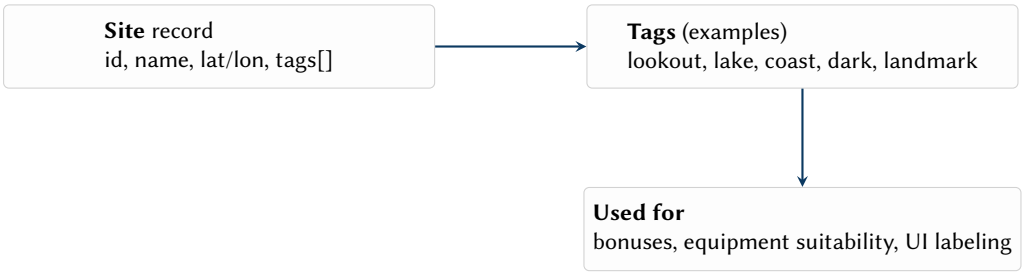


Figure 5. Site corpus modeling: minimal records, expressive tags.

5 UI and Map Integration

The application UI is implemented in `src/App.tsx`. State is persisted across sessions via a localStorage-backed hook for key parameters. Map rendering is handled by Mapbox GL:

1. A satellite basemap is loaded.
2. A GeoJSON source stores event arrow features.
3. Line layers render the current target direction, framing wedge edges, and event bearings.
4. DOM markers represent candidate sites; marker color reflects the current score.

5.1 Horizon Profile and Trajectory Overlay

When terrain elevation data is available, the app samples elevation along rays to estimate the maximum obstruction angle near the target azimuth. The algorithm is intentionally simple: for each azimuth in a small sweep window around the target, the app samples points along that ray, computes the maximum positive elevation difference relative to the observer point, and converts that to an obstruction angle. This yields a

horizon envelope that is sufficient to answer the practical question: *is the target likely to be blocked by nearby terrain at the moment it matters?*

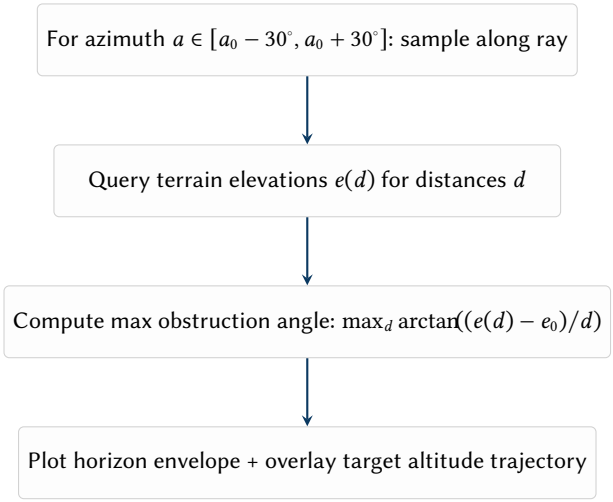


Figure 6. Horizon obstruction estimation pipeline used for the trajectory plot.

5.2 Field Execution Utilities

The UI includes:

1. Street View sync to import a scouted coordinate and approximate heading.
2. A live radar raster overlay via RainViewer tiles.
3. An itinerary panel that can be copied to clipboard as plaintext navigation links.

6 Build, Tooling, and Configuration

The project uses standard Vite scripts (`dev`, `build`, `lint`, `preview`). Full functionality requires a Mapbox token exposed as `VITE_MAPBOX_TOKEN`. Without a valid token, map rendering may still occur but terrain-dependent analysis (horizon profiling) is disabled. The application is therefore best understood as having two modes: *map-only* (ranking + bearings) and *terrain-aware* (ranking + bearings + obstruction profiling).

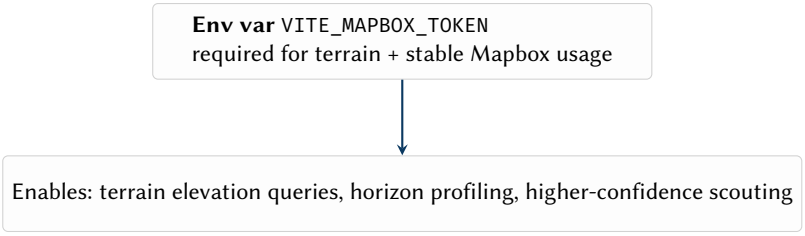


Figure 7. Configuration dependency: Mapbox token as a capability gate.

> **Design Decision 4: Token Safety**
>
> Hardcoded placeholder tokens must not be treated as production credentials. The configuration pathway should be explicit: missing token should produce a clear warning state rather than silently attempting degraded operation.

7 Limitations and Risks

The current implementation is effective as an exploratory planning tool but carries known limitations:

1. Event models include pragmatic simplifications (aurora proxy, meteor shower constants).
2. Light pollution and city glow are coarse; they capture "directionally correct" behavior, not calibrated photometry.
3. Marker lifecycle and map updates are optimized for simplicity rather than large-scale performance.
4. The optional `discovered.json` contract is not documented in-repo.

8 Next Steps

Engineering improvements that preserve the current product shape:

1. Surface score breakdowns in the UI and allow weight tuning.
2. Formalize configuration and document optional data files (`discovered.json`).
3. Improve astronomy fidelity where it drives decisions (time selection, trajectories).
4. Cache terrain samples and expose confidence/quality indicators.
5. Replace DOM markers with Mapbox data-driven layers for scalability.

References

[1] *Mapbox gl js*, https://docs.mapbox.com/mapbox-gl-js/, Accessed: 2026-03-11.
[2] *Astronomy-engine*, https://github.com/cosinekitty/astronomy, Accessed: 2026-03-11.
[3] *Vite*, https://vite.dev/, Accessed: 2026-03-11.
[4] *React*, https://react.dev/, Accessed: 2026-03-11.

List of Figures

List of Tables

Revision History

Version	Date	Changes
0.1.0	March 2026	Initial report extraction from current codebase